

Java

A Complete Guide

From Syntax to the Ecosystem

Contents

1	What Is Java?	3
2	How Java Works	3
3	Your First Java Program	3
4	Data Types	3
4.1	Primitive Types	4
4.2	Reference Types	4
5	Variables & Operators	4
6	Control Flow	4
6.1	If / Else	4
6.2	Switch (modern arrow syntax)	5
6.3	Loops	5
7	Object-Oriented Programming (OOP)	5
7.1	Classes & Objects	5
7.2	Encapsulation	6
7.3	Inheritance	6
7.4	Polymorphism	6
7.5	Abstraction	6
8	Access Modifiers	7
9	Key Keywords	7
10	Strings	7
11	Arrays & Collections	7
11.1	Arrays (fixed size)	7
11.2	ArrayList (dynamic size)	8
11.3	HashMap (key-value pairs)	8
12	Exception Handling	8
12.1	Throwing Exceptions	8
12.2	Custom Exceptions	8
13	Generics	9
14	Functional Programming (Java 8+)	9
14.1	Lambda Expressions	9
14.2	Streams	9
14.3	Optional	9
15	Multithreading & Concurrency	10
15.1	ExecutorService (Thread Pools)	10
16	File I/O	10
17	Important Java APIs & Libraries	11

18 Modern Java Features (Java 9–21+)	11
19 Best Practices	12
20 The Java Ecosystem	12
21 Learning Roadmap	12

1. What Is Java?

Java is a **general-purpose, object-oriented, platform-independent** programming language created by James Gosling at Sun Microsystems in 1995 (now owned by Oracle). Its famous motto is “*Write Once, Run Anywhere*” (**WORA**) — Java code compiles to **bytecode** that runs on any device with a **Java Virtual Machine (JVM)**.

2. How Java Works

```
Your Code (.java)
↓ Java Compiler (javac)
Bytecode (.class)
↓ JVM
Runs on any OS
```

- **JDK** (Java Development Kit) — tools to write and compile Java (includes JRE + javac).
- **JRE** (Java Runtime Environment) — runs Java programs.
- **JVM** (Java Virtual Machine) — interprets bytecode at runtime.

3. Your First Java Program

```
1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("Hello, World!");
4     }
5 }
```

Listing 1: Hello World in Java

Note

Every Java program lives inside a **class**. The main method is the entry point of every application. The filename *must* match the class name (HelloWorld.java).

4. Data Types

Java is **statically typed** — every variable has a fixed type.

4.1 Primitive Types

Type	Size	Example
byte	8-bit	byte b = 100;
short	16-bit	short s = 200;
int	32-bit	int x = 42;
long	64-bit	long l = 100L;
float	32-bit	float f = 3.14f;
double	64-bit	double d = 3.14;
char	16-bit	char c = 'A';
boolean	1-bit	boolean flag = true;

4.2 Reference Types

Objects, Arrays, and Strings are stored as references (pointers) to memory.

```
1 String name    = "Alice";      // String (immutable object)
2 int[] numbers = {1, 2, 3};    // Array
```

5. Variables & Operators

```
1 int a = 10, b = 3;
2
3 // Arithmetic
4 a + b    // 13
5 a - b    // 7
6 a * b    // 30
7 a / b    // 3 (integer division)
8 a % b    // 1 (modulo)
9
10 // Comparison
11 a == b   // false
12 a != b   // true
13 a > b    // true
14
15 // Logical
16 true && false // false (AND)
17 true || false // true  (OR)
18 !true        // false (NOT)
19
20 // Assignment shortcuts
21 a += 5;      // a = a + 5
22 a++;         // a = a + 1
```

6. Control Flow

6.1 If / Else

```
1 int score = 85;
2 if (score >= 90) {
3     System.out.println("A");
4 } else if (score >= 80) {
```

```
5     System.out.println("B");
6 } else {
7     System.out.println("C");
8 }
```

6.2 Switch (modern arrow syntax)

```
1 switch (day) {
2     case "Monday" -> System.out.println("Start of week");
3     case "Friday" -> System.out.println("End of week");
4     default       -> System.out.println("Midweek");
5 }
```

6.3 Loops

```
1 // For loop
2 for (int i = 0; i < 5; i++) {
3     System.out.println(i);
4 }
5
6 // While loop
7 int i = 0;
8 while (i < 5) { i++; }
9
10 // Do-While (runs at least once)
11 do { i++; } while (i < 5);
12
13 // Enhanced for-each
14 int[] nums = {1, 2, 3};
15 for (int n : nums) {
16     System.out.println(n);
17 }
```

7. Object-Oriented Programming (OOP)

OOP is Java's core paradigm, built on **four pillars**.

7.1 Classes & Objects

A **class** is a blueprint; an **object** is an instance of it.

```
1 public class Car {
2     // Fields (attributes)
3     String brand;
4     int speed;
5
6     // Constructor
7     public Car(String brand, int speed) {
8         this.brand = brand;
9         this.speed = speed;
10    }
11
12    // Method (behavior)
13    public void accelerate() {
14        speed += 10;
15    }
16 }
```

```
15         System.out.println(brand + " is now at " + speed + " km/h");
16     }
17 }
18
19 // Usage
20 Car myCar = new Car("Toyota", 60);
21 myCar.accelerate();    // Toyota is now at 70 km/h
```

7.2 Encapsulation

Hide internal state using private fields with public getters/setters.

```
1 public class BankAccount {
2     private double balance;
3
4     public double getBalance() { return balance; }
5
6     public void deposit(double amount) {
7         if (amount > 0) balance += amount;
8     }
9 }
```

7.3 Inheritance

A class can **extend** another, inheriting its fields and methods.

```
1 public class Animal {
2     public void speak() { System.out.println("..."); }
3 }
4
5 public class Dog extends Animal {
6     @Override
7     public void speak() { System.out.println("Woof!"); }
8 }
```

7.4 Polymorphism

The same method behaves differently based on the object type.

```
1 Animal a = new Dog();
2 a.speak();    // "Woof!" -- Dog's version is called
```

7.5 Abstraction

Hide complexity using **abstract classes** or **interfaces**.

```
1 // Interface -- a contract
2 public interface Shape {
3     double area();
4 }
5
6 public class Circle implements Shape {
7     double radius;
8     Circle(double r) { this.radius = r; }
9
10    @Override
```

```

11     public double area() { return Math.PI * radius * radius; }
12 }

```

8. Access Modifiers

Modifier	Same Class	Same Package	Subclass	Everywhere
private	✓	×	×	×
(default)	✓	✓	×	×
protected	✓	✓	✓	×
public	✓	✓	✓	✓

9. Key Keywords

Keyword	Purpose
static	Belongs to the class, not instances
final	Constant value / cannot be overridden
this	Refers to the current object
super	Refers to the parent class
new	Creates a new object
instanceof	Checks object type
abstract	Declares an abstract class/method
interface	Declares an interface

10. Strings

Strings in Java are **immutable objects**.

```

1 String s = "Hello, Java!";
2
3 s.length()           // 12
4 s.toUpperCase()      // "HELLO, JAVA!"
5 s.substring(0, 5)    // "Hello"
6 s.contains("Java")   // true
7 s.replace("Java", "World") // "Hello, World!"
8 s.split(", ")        // ["Hello", "Java!"]
9 s.trim()             // removes whitespace

```

For mutable string manipulation, use **StringBuilder**:

```

1 StringBuilder sb = new StringBuilder();
2 sb.append("Hello").append(", ").append("World!");
3 System.out.println(sb.toString());

```

11. Arrays & Collections

11.1 Arrays (fixed size)


```
1 int[] arr = new int[5];
2 String[] names = {"Alice", "Bob", "Carol"};
```

11.2 ArrayList (dynamic size)

```
1 import java.util.ArrayList;
2 ArrayList<String> list = new ArrayList<>();
3 list.add("Alice");
4 list.add("Bob");
5 list.remove("Alice");
6 list.get(0); // "Bob"
```

11.3 HashMap (key-value pairs)

```
1 import java.util.HashMap;
2 HashMap<String, Integer> map = new HashMap<>();
3 map.put("Alice", 30);
4 map.put("Bob", 25);
5 map.get("Alice"); // 30
6 map.containsKey("Bob"); // true
```

Note

Other useful collections include LinkedList, HashSet, TreeMap, Stack, and Queue.

12. Exception Handling

Java handles errors through **exceptions**.

```
1 try {
2     int result = 10 / 0; // throws ArithmeticException
3 } catch (ArithmeticException e) {
4     System.out.println("Cannot divide by zero: " + e.getMessage());
5 } catch (Exception e) {
6     System.out.println("General error: " + e.getMessage());
7 } finally {
8     System.out.println("This always runs.");
9 }
```

12.1 Throwing Exceptions

```
1 public void setAge(int age) {
2     if (age < 0) throw new IllegalArgumentException("Age cannot be
3         negative");
4     this.age = age;
5 }
```

12.2 Custom Exceptions

```
1 public class InsufficientFundsException extends Exception {
2     public InsufficientFundsException(String message) {
3         super(message);
4     }
5 }
```

```
4     }  
5 }
```

13. Generics

Generics allow classes and methods to work with **any type safely**.

```
1 public class Box<T> {  
2     private T value;  
3     public void set(T val) { this.value = val; }  
4     public T get()         { return value; }  
5 }  
6  
7 Box<String> strBox = new Box<>();  
8 strBox.set("Hello");  
9 String s = strBox.get();    // no cast needed
```

14. Functional Programming (Java 8+)

14.1 Lambda Expressions

```
1 // Old (anonymous class) way  
2 Runnable r1 = new Runnable() {  
3     public void run() { System.out.println("Hi"); }  
4 };  
5  
6 // Lambda expression  
7 Runnable r2 = () -> System.out.println("Hi");
```

14.2 Streams

Process collections in a declarative, pipeline style:

```
1 import java.util.List;  
2 import java.util.stream.*;  
3  
4 List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);  
5  
6 List<Integer> result = nums.stream()  
7     .filter(n -> n % 2 == 0)           // keep even numbers  
8     .map(n -> n * n)                   // square them  
9     .collect(Collectors.toList());    // [4, 16, 36]
```

14.3 Optional

Avoids NullPointerException:

```
1 Optional<String> opt = Optional.ofNullable(getName());  
2 opt.ifPresent(name -> System.out.println("Name: " + name));  
3 String name = opt.orElse("Unknown");
```

15. Multithreading & Concurrency

```
1 // Method 1: Extend Thread
2 class MyThread extends Thread {
3     public void run() { System.out.println("Running in thread"); }
4 }
5 new MyThread().start();
6
7 // Method 2: Implement Runnable (preferred)
8 Thread t = new Thread(() -> System.out.println("Lambda thread"));
9 t.start();
```

15.1 ExecutorService (Thread Pools)

```
1 import java.util.concurrent.*;
2 ExecutorService pool = Executors.newFixedThreadPool(4);
3 pool.submit(() -> System.out.println("Task 1"));
4 pool.shutdown();
```

Note

Use synchronized, volatile, and ReentrantLock to manage shared state safely across threads.

16. File I/O

```
1 import java.io.*;
2 import java.nio.file.*;
3
4 // Write to file
5 Files.writeString(Path.of("output.txt"), "Hello, File!");
6
7 // Read entire file
8 String content = Files.readString(Path.of("output.txt"));
9 System.out.println(content);
10
11 // Read line by line
12 Files.lines(Path.of("data.txt")).forEach(System.out::println);
```

17. Important Java APIs & Libraries

Category	Library / API
Collections	java.util.*
I/O & Files	java.io.*, java.nio.*
Networking	java.net.*
Date & Time	java.time.* (LocalDate, LocalDateTime)
Math	java.lang.Math
Regex	java.util.regex.*
HTTP Client	java.net.http.HttpClient (Java 11+)
JSON	Gson, Jackson
Database	JDBC, Hibernate
Web Frameworks	Spring Boot, Quarkus, Micronaut
Testing	JUnit 5, Mockito
Build Tools	Maven, Gradle

18. Modern Java Features (Java 9–21+)

Feature	Version	Example
Modules	9	module-info.java
var	10	var list = new ArrayList<>();
Text Blocks	15	"""multi-line string"""
Records	16	record Point(int x, int y) {}
Sealed Classes	17	sealed class Shape permits Circle, Square {}
Pattern Matching	21	if (obj instanceof String s) { ... }
Virtual Threads	21	Lightweight concurrency via Project Loom

19. Best Practices

Best Practice

- Follow naming conventions: camelCase for variables/methods, PascalCase for classes, UPPER_SNAKE_CASE for constants.
- Prefer **composition over inheritance** where possible.
- **Program to interfaces**, not implementations.
- Always close resources using **try-with-resources**:

```
1 try (BufferedReader br = new BufferedReader(new FileReader("file.  
2     txt"))) {  
3     // br is automatically closed  
4 }
```

- Avoid null — use Optional instead.
- Use final for variables that should not change.
- Write **unit tests** with JUnit.
- Use a **build tool** (Maven or Gradle) for dependency management.

20. The Java Ecosystem

Java SE → Core language (what this guide covers)
Java EE → Enterprise apps (Servlets, JPA, ...) -- now Jakarta EE
Android → Mobile development (Java / Kotlin)
Spring → The dominant web framework ecosystem
Maven/Gradle → Build & dependency management

21. Learning Roadmap

1. **Beginner:** Syntax → OOP → Arrays / Collections → Exception Handling
2. **Intermediate:** Generics → Streams → File I/O → Multithreading
3. **Advanced:** Design Patterns → Spring Boot → Databases (JDBC / JPA) → Microservices
4. **Expert:** JVM Internals → Performance Tuning → Distributed Systems

Java

Java is a deep and rich language — but its consistent design means that once you internalize OOP and the standard library, everything else builds naturally on top.

Happy coding!